RPi 4B tests — build, run, clean (µOS++ III, AArch64)

Every command here was run on fresh clones in `/tmp` (2026-10-13): one from GitHub and one from the local bare repos in `~/Downloads/GIT`. Nothing depends on the folder you clone into; see §8.

0. Prerequisites (once per PC)

Need	Command / note
`git`, Node.js + `npm`	from the distribution (tested: git 2.56, node 25.8, npm 11.11)
`xpm`	`npm install -g xpm@latest` (tested: 0.23.3)
internet	the first `xpm install` downloads CMake, Ninja, GCC and QEMU into `~/.local/xPacks`
OpenOCD, for `-hwd` only	`xpm install -g @xpack-dev-tools/openocd@0.12.0-7.1`
probe, for `-hwd` only	CMSIS-DAP JTAG probe with user access (udev); a USB-serial on the Pi UART (GPIO14/15)

The rpi4b configurations do **not** install OpenOCD. The board runner uses the newest OpenOCD in `~/.local/xPacks/@xpack-dev-tools/openocd/`, otherwise `openocd` from `PATH`, or the binary named in `OPENOCD` (§6). Without one of these, every `-hwd` test fails. All board runs in §8 used 0.12.0-7.1.

1. Get the sources (any folder)

1a. From GitHub (`github.com/dan-maio`)

```
W=/any/folder                       # e.g. ~/Work, /tmp/rpi4b-verify
mkdir -p "$W" && cd "$W"
for r in micro-os-plus-iii micro-os-plus-iii-aarch64 micro-os-plus-iii-aarch32 \
         micro-os-plus-iii-cortexm micro-os-plus-iii-posix-arch; do
  git clone -b smp https://github.com/dan-maio/$r.git $r
done
cd "$W/micro-os-plus-iii/tests"
npm install                         # once per clone
xpm install                         # once per clone: CMake, Ninja, build-helper
```

- On GitHub the devices drivers are already absorbed: `micro-os-plus-iii-aarch64/drivers/{src,include,fatfs}` are real folders. So there is no devices clone.

1b. From the local bare repos (`~/Downloads/GIT`)

```
W=/any/folder                       # e.g. /tmp/rpi4b-local
G=~/Downloads/GIT
mkdir -p "$W" && cd "$W"
git clone -b smp $G/micro-os-plus-iii-smp.git micro-os-plus-iii
for r in aarch64 aarch32 cortexm posix-arch; do
  git clone -b smp $G/micro-os-plus-iii-$r.git micro-os-plus-iii-$r
done
git clone -b master $G/micro-os-plus-iii-devices.git micro-os-plus-iii-devices   # required here
cd "$W/micro-os-plus-iii/tests"
```

```
npm install
xpm install
```

- **`micro-os-plus-iii-devices` is required here.** The local aarch64 port has three committed symlinks, `drivers/{src,include,fatfs} -> ../../micro-os-plus-iii-devices/…`.
- Without the devices clone, `prepare` fails with `Cannot find source file: …/micro-os-plus-iii-aarch64/drivers/src/sd.cpp`, nothing builds, and every test fails.
- Note the names: the smp repo is `micro-os-plus-iii-smp.git`, and it is cloned as `micro-os-plus-iii`.

Both ways

- Use a folder path without spaces (paths with spaces were not tested with CMake).
- The repositories must sit **side by side** in `$W`. CMake finds the ports as `../../micro-os-plus-iii-<port>`, with or without a `.git` suffix.
- Everything after this section is the same for both.

2. Names

Item	Value
Run all commands from	`$W/micro-os-plus-iii/tests`
Test sources	`$W/micro-os-plus-iii-aarch64/test/rpi4b/<test>/`
Board files (OpenOCD cfg, `hw.sh`)	`$W/micro-os-plus-iii-aarch64/test/boards/rpi4b/`
Configurations `C`	`aarch64-rpi4b-cmake-gcc-debug` (Debug), `aarch64-rpi4b-cmake-gcc-release` (MinSizeRel)
Build folder	`build/$C/` (one per configuration)
Images	`build/$C/platform-bin/port-tests/test/<test>-hwd.elf`, `<test>-qemu.elf`
CTest case	`aarch64-rpi4b-<test>-<variant>`

Variants

- `-qemu` runs in QEMU (`raspi4b`); needs no board.
- `-hwd` runs on the board: OpenOCD over the CMSIS-DAP JTAG probe, with no GDB.

Tests: `smp_test0` … `smp_test4`, `smp_mem_test`, `smp_mem_inter_test`, `smp-mat-test`, `smp-mat-sdcard-test`, `smp-num-test`, `smp-pipeline-test`, `smp-pro-cons-test`, `sd_test`, `cmsis-os-validator`, `mutex-stress`, `rtos-apis`, `smp-list-storage-test`, `smp-shapes-test`, `clock-test`.

The last two are the C++ versions of tests first written for the Zig port, check for check: `smp-list-storage-test` is a generic intrusive list with its nodes on thread stacks, in `.data` and on the heap, under thread churn on the 4 cores (UART only); `smp-shapes-test` is a list of polymorphic shapes (an abstract C++ base class and four derived ones, built and destroyed by threads on the 4 cores), with a logger thread writing every line to the UART and to `tests/shapes.log` on the SD card, read back and CRC-checked. Both passed on the board on 2026-10-22 (35 s and 125 s).

`clock-test` checks the kernel's clocks against the BCM2711 System Timer (1 MHz): 5000 ticks of `sysclock`, and `hrclock` over them, keep its time; `hrclock.sleep_until (now () + 100 ms)` lasts 100 ms (+1 tick) on core 0 and on core 1; and a thread on core 1 wakes promptly after core 0's tick. It guards the clock fixes of 2026-10-22: the tick re-armed on an absolute grid (`timer_arm::rearm_1ms()`, `CNTP_CVAL += period`; re-arming one period from *now* in the interrupt lost the handler's latency every tick), `hrclock.now()` counted from core 0's tick origin (`timer_arm::hr_base()`; it was the raw counter since power-on, while its sleeps count from the tick's start), and the counter's frequency calibrated once instead of by every core. On the board: 5000 ticks = 5 000 000 µs, the

100 ms sleep 101.0 ms on both cores (11 s). The same test exists for the Pico 2 and for the Zero 2 W and Pi 3 B of both ports.

```
cd "$W/micro-os-plus-iii/tests"
C=aarch64-rpi4b-cmake-gcc-debug        # or aarch64-rpi4b-cmake-gcc-release
T=smp_mem_test                         # the test of your choice
```

3. First time for a configuration (in this order)

```
xpm install --config $C          # pinned aarch64 GCC 15.2.1 and QEMU 9.2.4 -> build/$C/xpacks
        (not OpenOCD, §0)
xpm run prepare --config $C      # CMake configure; finds the test folders
```

- Debug and release are separate folders, so each needs both steps.
- Use `xpm install --config $C`, **not** `xpm run install`. The action adds an `xpm link @micro-os-plus/micro-os-plus-iii-aarch64` step, which nothing uses: CMake takes the port from the sibling folder. That step fails when the machine has a stale global link in `~/.local/xPacks` (`error: broken link …`). See `docs/tests/XPM-LINK-VS-SIBLING-LAYOUT.md` §3.4.
- Never `rm -rf build/$C`, because it holds the pinned toolchain.

4. Build

Goal	Command
all images of the configuration	`xpm run build --config $C`
one image only	`PATH="$PWD/build/$C/xpacks/.bin:$PATH" cmake --build build/$C --target $T-hwd` (or `$T-qemu`)
list the test cases	`PATH="$PWD/build/$C/xpacks/.bin:$PATH" ctest --test-dir build/$C -N`

After you add or rename a test folder, run `xpm run prepare --config $C` again.

5. Run

One test on the board. One at a time, with the UART terminal open:

```
UOS_HW_ADAPTER_KHZ=15000 xpm run test-$T-hwd --config $C
```

This runs prepare and build, then `ctest -V -R aarch64-rpi4b-$T-hwd`. To run without rebuilding:

```
PATH="$PWD/build/$C/xpacks/.bin:$PATH" UOS_HW_ADAPTER_KHZ=15000 \
ctest --test-dir build/$C -V -R aarch64-rpi4b-$T-hwd
```

One test in QEMU:

```
xpm run test-$T-qemu --config $C
```

Sets:

Goal	Command
all QEMU tests (does not build)	`xpm run test --config $C` (= `ctest -V -LE hwd`)
all board tests, back to back (**not verified**, see §8)	`UOS_HW_ADAPTER_KHZ=15000 xpm run test-hwd --config $C`

6. Board settings

Variable	Default	Meaning
`UOS_HW_ADAPTER_KHZ`	`4000`	JTAG TCK after connect; `10000`–`15000` tested with the PIO probe firmware
`UOS_HW_RESET_WAIT`	`10`	seconds after the watchdog reset, for the firmware to boot and re-enable JTAG
`OPENOCD`	newest xPack, else `PATH`	the OpenOCD binary to use, e.g. `~/.local/xPacks/@xpack-dev-tools/openocd/0.12.0-7.1/.content/bin/openocd`

What one `-hwd` run does:

1. Resets the board through the BCM2711 watchdog, so no power cycle is needed between tests.
2. Waits `UOS_HW_RESET_WAIT` seconds.
3. Connects at 4 MHz, then switches to `UOS_HW_ADAPTER_KHZ`.
4. Loads the image at `0x80000` and starts core 0. The kernel releases cores 1–3.
5. Waits for `RESULT: PASS|FAIL` over semihosting.

The full console goes to the UART only (GPIO14/15, 115200 8N1). When it connects, four `Failed to read memory at 0x80{4..7}10088` errors appear. They are normal.

7. Clean

Goal	Command
remove the build outputs (keeps toolchain and CMake config)	`xpm run clean --config $C`
force a fresh CMake configure	`rm build/$C/CMakeCache.txt && xpm run prepare --config $C`

Results

- **Pass:** `RESULT: PASS`, followed by `100% tests passed`.
- **Fail:** `RESULT: FAIL` or a timeout. Read the `ctest -V` output, then the UART.

8. Verification (GitHub clone in `/tmp/rpi4b-verify`, 2026-10-13)

The five repositories were cloned from `github.com/dan-maio` (branch `smp`) into `/tmp`, with no link to any other workspace.

Command	Debug	Release
`xpm install --config $C`		
`xpm run prepare --config $C`		
one image (`cmake --build … --target $T-hwd`)		
`xpm run build --config $C`		
`ctest -N` (list)		
`xpm run test --config $C` (all QEMU)	16/16 passed, 212 s	16/16 passed, 212 s

Command	Debug	Release
`xpm run test-$T-qemu --config $C`		not run alone (passed within the 16/16)
`xpm run clean --config $C`	images removed	images removed
`xpm run build --config $C` after clean		
`UOS_HW_ADAPTER_KHZ=15000 xpm run test-smp_test2-hwd --config $C`	`RESULT: PASS`, 14 s	—
`xpm run install --config $C`, as on GitHub	fails at the unused `xpm link` step (§3)	—
`xpm run install --config $C`, with the `xpm link` step removed from `aarch64-actions` (local edit in the `/tmp` clone only, not on GitHub)		

Second run: unmodified GitHub clone (2026-10-13)

`/tmp/rpi4b-verify` was deleted and cloned again from GitHub, with no file changed. Only the commands of §1–§5 were used, with `xpm install --config $C` (not `xpm run install`):

Command	Debug	Release
`npm install`, `xpm install`		
`xpm install --config $C`		
`xpm run prepare --config $C`		
`xpm run build --config $C`	32 images	32 images
`xpm run test --config $C` (all QEMU)	16/16 passed, 212 s	16/16 passed, 211 s
`UOS_HW_ADAPTER_KHZ=15000 xpm run test-smp_test2-hwd --config $C` (board)	`RESULT: PASS`, 14 s	—

Conclusion: the GitHub repositories install, build and test (QEMU and board) correctly as they are. The only failing command is `xpm run install`, because of its unused `xpm link` step (§3).

Third run: clone from `~/Downloads/GIT` (2026-10-13)

The six repositories were cloned as in §1b into `/tmp/rpi4b-local`, with no file changed. That is the same commits as the golden workspace, all committed and pushed.

Command	Debug	Release
without the devices clone: `xpm run prepare --config $C`	`Cannot find source file … drivers/src/sd.cpp`	same
`npm install`, `xpm install`		
`xpm install --config $C`		
`xpm run prepare --config $C`		
`xpm run build --config $C`	32 images	32 images
`xpm run test --config $C` (all QEMU)	16/16 passed, 212 s	16/16 passed, 212 s

Command	Debug	Release
board `smp_test3-hwd`, 15 MHz (`ctest … -R aarch64-rpi4b-smp_test3-hwd`)	`RESULT: PASS`, 24 s	—
board `smp_test2-hwd`, 15 MHz (`xpm run test-smp_test2-hwd`)	`RESULT: PASS`, 14 s	—

Conclusion: with the devices clone, the `~/Downloads/GIT` repositories install, build and test (QEMU and board) correctly as they are.

Fourth run: the guide's own commands, verbatim (2026-10-13)

The `sh` blocks of this file were extracted unchanged and run as one script with `set -e`, in a new folder `/tmp/rpi4b-doc`. Only `W=` was set, and `T=smp_test2` before the board command. The script ran §1a, §2, §3, §4 `build`, §5 `test`, §5 `test-$T-qemu` and the §5 board command without rebuild.

Step (debug)	Result
§1a: 5× `git clone`, `npm install`, `xpm install`	
§3: `xpm install --config $C`, `xpm run prepare --config $C`	
§4: `xpm run build --config $C`	
§5: `xpm run test --config $C`	16/16 passed
§5: `xpm run test-smp_mem_test-qemu --config $C`	
§5: `ctest … -R aarch64-rpi4b-smp_test2-hwd`, 15 MHz	`RESULT: PASS`, 14 s

Golden workspace: `smp_mem_inter_test-hwd` at 15 MHz passed in 525 s.

What was not verified

- `xpm run test-hwd` (all 16 board tests back to back); only single board tests were run.
- A PC without OpenOCD in `~/.local/xPacks`. This PC already had it, so §0's `xpm install -g` line was checked only as a no-op ("already installed").
- `xpm run install` on a PC with **no** global `micro-os-plus-iii-aarch64` link. It is expected to fail too (nothing to link), which is why §3 uses `xpm install --config $C`.